

Lab03

Bringup and control a real robot

Objectives.....	1
Requirements.....	1
Bringup Turtlebot3.....	1
Visualize robot state and sensors with Rviz2.....	3
Teleoperation nodes.....	3
Power the motors.....	3
Exercise.....	6
Appendix.....	7
General Information.....	7
Secure Shell Connection (command line).....	7
VS Code Remote Development (edit code).....	7
ROS 2 Connection.....	8
Robot checklist.....	9

Objectives

- Start Turtlebot3 driver and communicate with robot and sensors
- Drive a real robot with velocity commands
- Drive a real robot with the **Bump&Go controller** developed in simulation

Requirements

Create a workspace for your group in the home folder named `~/group_NNN`, where NNN is the number of your group. **Do not try to modify files inside `~/tb3_ws`**, this is a **read-only** folder that contains the main packages for running Turtlebot3 and its simulation. Create a copy in your own workspace if you need a modified version of these packages.

Bringup Turtlebot3

In robotics, "**bringup**" refers to the initial process of setting up and configuring a robot's hardware and software components connecting sensors, actuators, and controllers, as well as loading the necessary firmware and software stacks.

In this section there is a description of the main checks and commands to bring up the Turtlebot3.

Check the Sections [Secure Shell Connection \(command line\)](#) and [VS Code Remote Development \(edit code\)](#) to understand how to connect to the robot shell.

Check the `~/ .bashrc` content (on robot PC)

Since the robot will be used by different groups during the weeks, it is suggested as **best practice** to open the `~/ .bashrc` file **on the robot PC** and **check its content**.

- ☐ Verify the exported **environment variables**. Here it is reported a list of the necessary ones.
 - ☐ `RMW_IMPLEMENTATION=rmw_zenoh_cpp`
 - ☐ `TURTLEBOT3_MODEL=burger`
 - ☐ `LDS_MODEL=LDS-02`
 - ☐ `ROS_DOMAIN_ID=X`, where X is the number of the robot. In this case do not put the 0 in front if it has only one digit.
 - ☐ `CAMERA_MODEL=realsense` (or `oakd`). Only if the lab requires the use of a camera, otherwise do not set it.
- ☐ Verify the **sourced workspaces**. Here are the minimum checks that you shall perform when starting a new lab.
 - ☐ Source `/opt/ros/humble/setup.bash`
 - ☐ Source `/opt/ros/tb3_ws/install/setup.bash`
 - ☐ Make a **source** command for the **workspace of your group**
 - ☐ **Comment** with an `#` at the start of the line every other source to **workspaces of other groups**

Start the Turtlebot3 nodes (on robot PC)

Make sure the `rmw_zenohd` node is running, but do not start more than one of them.

```
ros2 run rmw_zenoh_cpp rmw_zenohd
```

All the software necessary to communicate with the robot and the sensors is started with a **single launch**. Run the following command in an SSH shell:

```
ros2 launch turtlebot3_bringup robot.launch.py
```

You can **check** that the system is running by searching for these lines in the logs

```
[ld08_driver-2] LDS-02 started successfully
...
[turtlebot3_ros-3] [INFO] [1730373263.920085681] [turtlebot3_node]: Run!
[turtlebot3_ros-3] [INFO] [1730373263.923334200] [diff_drive_controller]: Init Odometry
[turtlebot3_ros-3] [INFO] [1730373263.928420518] [diff_drive_controller]: Run!
```

Now, if you run the command `ros2 topic list` you should see a list of topics similar to the one you saw in simulations.

Troubleshooting

- If you get an error in the connection with the port `/dev/ttyACM0`, you shall run the command `sudo usermod -aG dialout students` and reboot the PC.

- If you do not see any topic when running `ros2 topic list` try to run the following command, then try again.

```
pkill -9 -f ros && ros2 daemon stop
```

Visualize robot state and sensors with Rviz2 (on your PC)

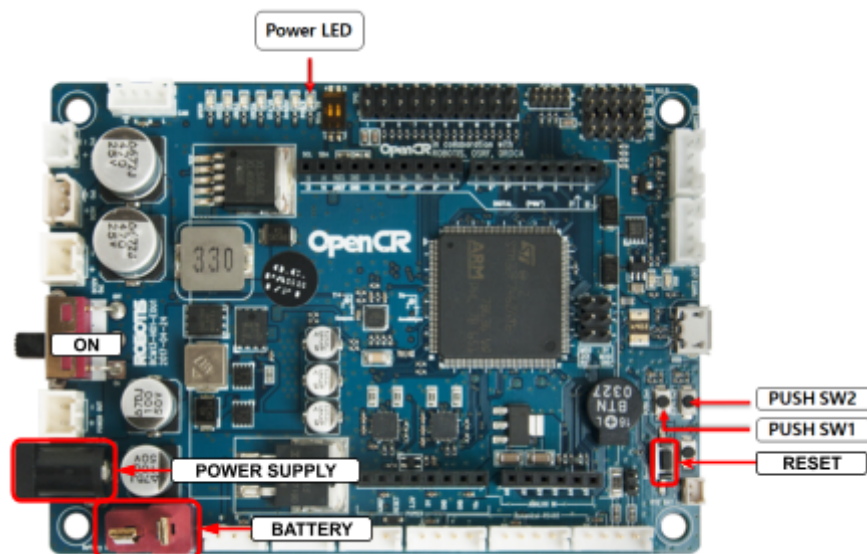
1. Follow the procedure reported in [Appendix](#) for [ROS 2 Connection](#)
2. While the bringup command is running, open Rviz2 on your PC. Now, you should see the laser scan points and the robot model.

```
ros2 launch turtlebot3_bringup rviz2.launch.py
```

Power the motors

Power supply: 12V DC, or Lithium battery 3S (11.1 V)

- **AT YOUR DESK:**
 - Make sure the **robot is raised** from the table and the wheels are not in contact with the ground
 - **Plug** the **12V** power supply
 - Move the switch to the **ON position**, the Power LED should turn green and a music should play after some seconds
 - To **verify** that everything is working properly you can keep pressing SW1 or SW2 for some seconds, the wheels should move
 - If the bringup is started, now you can command the robot from ROS 2 topics.
- **ON TEST FIELD:**
 - **Plug** the Lithium battery
 - Move the switch to the **ON position**, the Power LED should turn green and a music should play after some seconds.
 - If the bringup is started, now you can command the robot from ROS 2 topics.



Teleoperation nodes

While the bringup is running, you can also send a velocity command to the robot to make it move. In the following sections, **three different methods** to send velocity commands are shown.

a. ROS 2 cli

One simple way of sending a velocity command is publishing a velocity on the `/cmd_vel` topic through the command line interface. The `-r` option is used to control the publication frequency in Hz.

```
ros2 topic pub /cmd_vel geometry_msgs/msg/Twist -r 1 "linear:
  x: 0.0
  y: 0.0
  z: 0.0
angular:
  x: 0.0
  y: 0.0
  z: 0.0"
```

b. Keyboard teleoperation

Another interface that can be used from a terminal is the `teleop_keyboard` node of the `turtlebot3_teleop` package. This node allows you to change the velocity in real time using your keyboard. Read the instructions on the terminal after starting the node.

```
ros2 run turtlebot3_teleop teleop_keyboard
```

c. rqt Robot Steer

This is a graphical interface to send velocity commands. You can run it with the following command, or by opening `rqt` and searching in the *Plugins > Robot Tools* section.

```
ros2 run rqt_robot_steering rqt_robot_steering
```

Set the topic name in the top left text box. Then set the maximum and minimum velocities in the text boxes, according to the robot limits. Now, you can use the sliders to set the desired velocity.

If the plugin is not installed, run the following command

```
sudo apt update && sudo apt install ros-humble-rqt-robot-steering
```

Exercise

Modify the controller you developed in Lab2 to make it work on the real robot:

- Change the Quality of Service (QoS) of the subscription to the topic `/scan`

`# from this`

```
self.create_subscription(LaserScan, "scan", self.laser_callback, 10)
```

`# to this`

```
from rclpy.qos import qos_profile_sensor_data
```

```
self.create_subscription(LaserScan, "scan", self.laser_callback,  
qos_profile_sensor_data)
```

- Change the way in which you select the points from the array of ranges:
 - On the real sensor, due to hardware specifications, it is not guaranteed that you will have 360 values in the ranges field. Use the fields `angle_min`, `angle_max` and `angle_increment` to select the ranges in the correct direction.

Try the controller you developed in Lab2 on the real Turtlebot3.

Record a rosbag while the robot is moving and try to **plot** the data as suggested in Lab2. Does the odometry resemble the real path of the robot? Could you say why?

Appendix

General Information

Robot PC

IP Address: 192.168.10.1XX, where XX is the number of the robot

Hostname: nucXX, where XX is the number of the robot

Username: students

Password: sesasr

WiFi network

SSID: SESASR_WiFi

Password: LED05_2024

Secure Shell Connection (command line)

SSH (Secure Shell) is a cryptographic network protocol used to securely connect to remote computers over an unsecured network. It provides encrypted communication for tasks such as remote login, file transfers, and command execution. SSH ensures that data, authentication credentials, and commands are protected from eavesdropping or tampering. Commonly used for system administration, it typically operates on port 22 and uses public-key cryptography or password-based authentication.

You will use SSH to open remote terminals on the robot PC from your computer.

1. Connect to the SESASR_WiFi network
2. Open a terminal (PowerShell, WSL2, and Ubuntu will work in the same way)
3. Use the following command to open a terminal on the remote PC

```
ssh students@192.168.10.1XX
```

4. At the first connection you will see the following prompt, reply **yes**

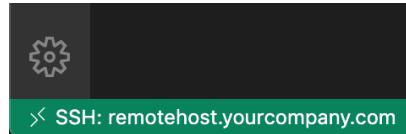
```
The authenticity of host 'nucXX (192.168.10.1XX)' can't be established.  
ED25519 key fingerprint is  
SHA256:vqE1rQf6k08I4/P+KxsvyglQGHphutzB1fgmi3BwVPs.  
This key is not known by any other names.  
Are you sure you want to continue connecting (yes/no/[fingerprint])?
```

5. Now, you are logged in. Run the commands as if you were in a shell of the robot PC. You can open as many SSH connections as you need, e.g. one for each node/launch that you need to run.

VS Code Remote Development (edit code)

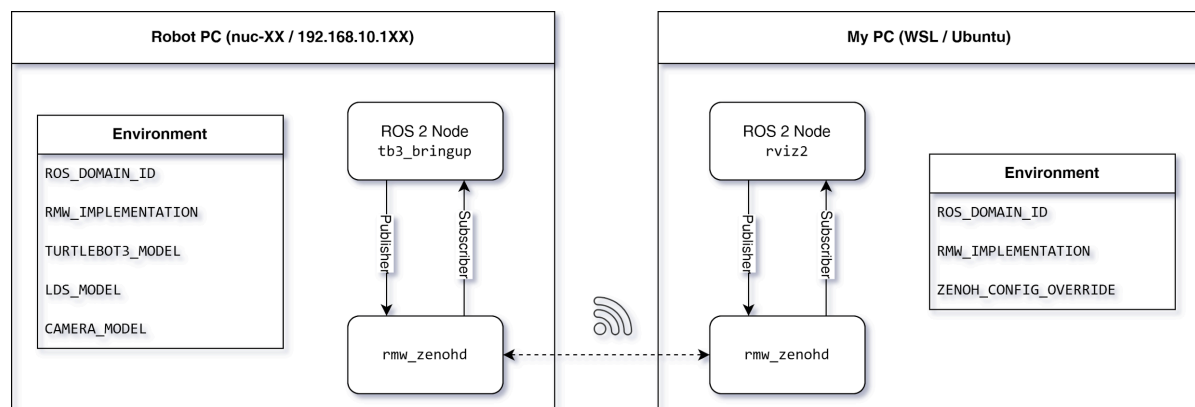
1. Open VS Code and install "Remote - SSH" extension
2. From the **Command Palette** (F1, Ctrl+Shift+P, View>Command Palette...), select **Remote-SSH: Connect to Host...**
3. Put students@192.168.10.1XX when asked in the input box

4. Select the Operating System of the host PC (Linux)
5. VS Code will take some time for the first setup. After the connection is established, you should see something similar to this in the bottom left corner.



6. Now you can work as you were on your computer

ROS 2 Connection



In order to visualize the topics published by the Turtlebot3 on your computer (e.g. to use Rviz2) you shall follow these steps **ON YOUR COMPUTER**.

It is suggested to put the following lines in the `~/ .bashrc`, so you do not need to run these commands every time you open a new shell. Otherwise, run these lines before every command reported below. **Substitute XX** in the IP address and domain id with your Turtlebot3 number.

```
export ROS_DOMAIN_ID=X
export RMW_IMPLEMENTATION=rmw_zenoh_cpp
export ZENOH_CONFIG_OVERRIDE='mode="client";connect/endpoints=["tcp/192.168.10.1XX:7447"]'
```

1. Install ROS middleware Zenoh (**only the first time**)

```
sudo apt update && sudo apt install ros-humble-rmw-zenoh-cpp
```

2. In one terminal run the following command to start the Zenoh router. **Keep this terminal open**, since the `rmw_zenohd` node shall be always active.

```
ros2 run rmw_zenoh_cpp rmw_zenohd
```

3. In another terminal, run the commands `ros2 topic list` and you should see all the topics published by the Turtlebot3.

```
ros2 topic list
```

4. You can also run `rviz2` to visualize sensor data, such as LiDAR.

Robot checklist

You can print this checklist and bring it with you to laboratories.

Open the `~/ .bashrc` file and perform all the following checks.

- ☐ **Verify** the exported **environment variables**. Here it is reported a list of the necessary ones.
 - ☐ `RMW_IMPLEMENTATION=rmw_zenoh_cpp`
 - ☐ `TURTLEBOT3_MODEL=burger`
 - ☐ `LD_LIBRARY_PATH=/opt/ros/humble/lib`
 - ☐ `ROS_DOMAIN_ID=X`, where X is the number of the robot. In this case do not put the 0 in front if it has only one digit.
 - ☐ `CAMERA_MODEL=realsense` (or `oakd`). Only if the lab requires the use of a camera, otherwise do not set it.
- ☐ **Verify** the **sourced workspaces**. Here are the minimum checks that you shall perform when starting a new lab.
 - ☐ Source `/opt/ros/humble/setup.bash`
 - ☐ Source `/opt/ros/tb3_ws/install/setup.bash`
 - ☐ Put a **source** command for the **workspace of your group**
 - ☐ **Comment** with an `#` at the start of the line every other source to **workspaces of other groups**
- ☐ **In order to move the robot**
 - ☐ In a terminal 1 start `robot.launch.py`
 - ☐ In terminal 2 start a control node, e.g. `teleop_keyboard` or a node written by you.

Troubleshooting

- If you get an error in the connection with the port `/dev/ttyACM0`, you shall run the command `sudo usermod -aG dialout students` and reboot the PC.
- If you do not see any topic when running `ros2 topic list` try to run the following command, then try again.

```
killall -9 -f ros && ros2 daemon stop
```